

S. Nithyashree
192511007

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 25

Duration :60 Mins
On Class
Total Marks:50

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I S. Nithyashree / 192511007 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1) The Parser checks whether the token sequence $id + id * id$ follows the grammar rules. It gives $*$ higher precedence than $+$, so the expression is parsed as: $id + (id * id)$.

2) Grammar: $S \rightarrow aSb \mid \epsilon$

Leftmost derivation:

S
 $\rightarrow aSb$
 $\rightarrow aaSbb$
 $\rightarrow aaaSbbb$
 $\rightarrow aaaa bbbb$

Hence, $aaaabbbb$ belongs to the language.

3) Parser left recursion:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

4) Grammar:

$A \rightarrow aB \mid \epsilon$

$B \rightarrow b$

$\text{First}(A) = \{a, \epsilon\}$

5) Grammar:

$A \rightarrow Aa \mid b$

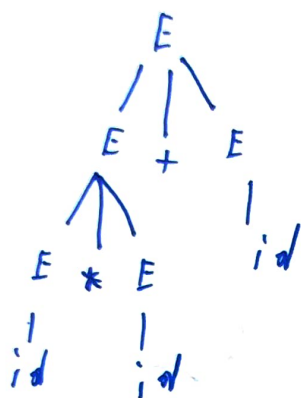
Not suitable for top-down parsing because it contains left recursion.

6) Left factoring :

$S \rightarrow \text{while } E. \text{ do } SX$

$X \rightarrow \text{else } S/E$

7) Parse tree :



8) Recursive descent Procedures :

void S() {

 match ('a');

 A();

}

void A() {

 if (input == 'b') {

 match ('b');

 A();

 }

 else if (input == 'c')

 {

 match ('c');

9) Predictive Parsing Table :

Non-Terminal	a	b	\$
S	$S \rightarrow AB$	$S \rightarrow AB$	
A	$A \rightarrow a$	$A \rightarrow \epsilon$	
B		$B \rightarrow b$	

10) Shift-Reduce Parser (first three actions) :

1. shift id
2. Reduce $id \rightarrow \epsilon$
3. shift +

11) Handle : For $id * id + id$, the first handle is id.

12) operator Precedence :

$* > +$

Relations :

$+ < *$

$* > +$

$+ = +$

$* = *$

13) $id + id * id$

operator Precedence Parser evaluates :

$id * id$

first, then

$id + (id * id)$

- 14) LR Parser : An LR Parser takes input left-to-right and produces the rightmost derivation in reverse.

Ex :

$id + id$

shift id

Reduce $E \rightarrow id$

shift $+$

shift id

Reduce $E \rightarrow id$

Reduce $E \rightarrow E + E$

Accept

- 15) Steps for constructing an SLR table :

1. Augment grammar.
2. Construct LR(0) items.
3. Compute closure.
4. Compute GOTO.
5. Find First and Follow.
6. Fill Action table.
7. Fill GOTO table.

16) Grammar :

$$S \rightarrow AB$$

$$A \rightarrow a \mid \epsilon$$

$$B \rightarrow b$$

$$\text{Follow}(A) = \{b\}$$

17) Augmented grammar :

$$S' \rightarrow S$$

$$S \rightarrow aA$$

$$A \rightarrow b$$

18) Closure :

$$S' \rightarrow \bullet S$$

$$S \rightarrow \bullet aA$$

19) Grammar :

$$E \rightarrow E + E \mid id$$

String :

$$id + id + id$$

The grammar is ambiguous because it has multiple parse tree.

20) Leftmost derivation :

E

$$\rightarrow E + E$$

$$\rightarrow id + E$$

$$\rightarrow id + E * E$$

$\Rightarrow id + id * E$

$\Rightarrow id + id * id$

21) Left factoring :

$S \rightarrow aBx \mid b$

$x \rightarrow c \mid d$

22) Grammar :

$F \rightarrow (E) \mid id$

$First(F) = \{ (, id \}$

23) Grammar :

$S \rightarrow aA \mid aB$

$A \rightarrow b \quad B \rightarrow c$

Not suitable for Predictive Parsing because both productions start with a , causing First set conflict.

24) Recursive descent parser :

void $S()$ {

if (input == 'a') {

match('a');

match('b');

$S()$;

}

(Else {

25) Non terminal a b $\$$

S

$S \rightarrow aS$

$S \rightarrow b$